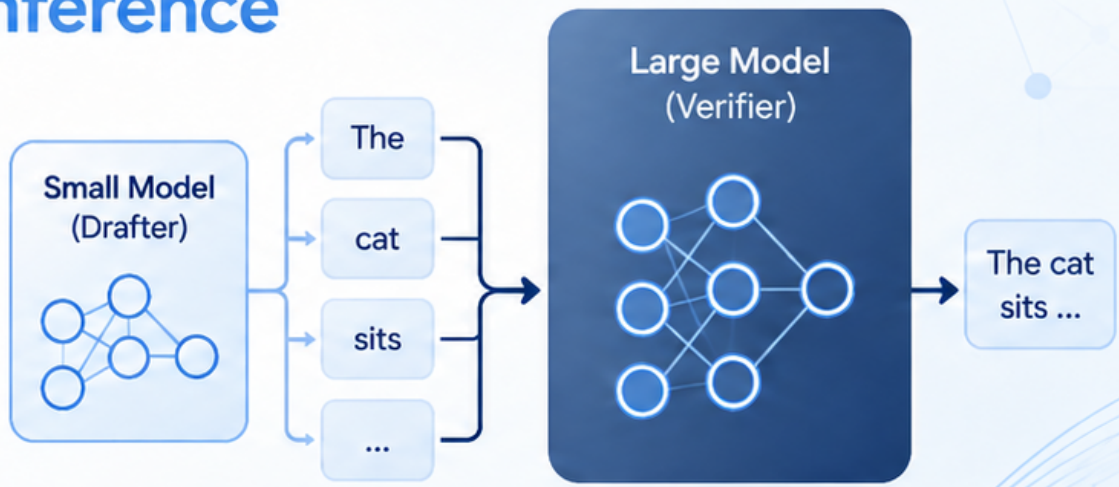
Speculative Decoding

Machine Translation and Multilingual Inference

A small model drafts text for a larger model to verify, aiming to reduce decoding time.



Presenter: Shaik Kashif
Sec: AIML C3
PRN: 24070126215

Understanding of the Topic

Definition

A small draft model proposes several tokens. A larger target model checks them together and keeps an agreed prefix.

1 What problem does it solve?

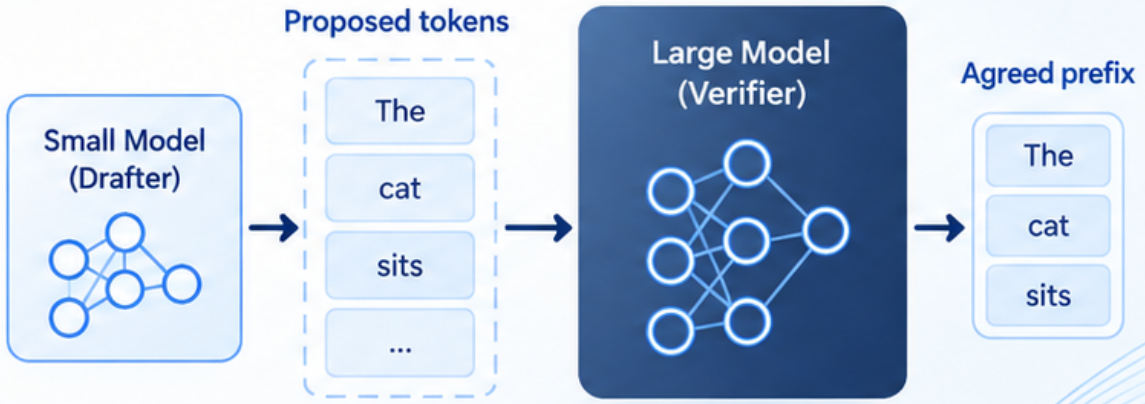
Normal decoding needs one sequential target-model call per new token.

2 Where does it fit?

Transformer inference, advanced NLP and neural machine translation.

3 Why should the audience care?

It can reduce translation and response latency when drafting is cheap and agreement is high.



Example

Scenario

Published translation case

- English to German: WMT EnDe
- Target: T5-XXL (11B parameters)
- Draft: T5-small (77M parameters)
- Goal: lower translation decoding latency

My text-generation experiment

- Qwen2.5: 3B target + 0.5B draft
- RTX 4060 Laptop GPU; FP16
- 10 questions/language; 2 repeats
- 300 trials; up to 128 new tokens

Illustrative translation example

(not measured model output)

Source: The cat sleeps

Draft: Die Katze sitzt

Target: Die Katze schläft.

What Happens / Result

Published translation result: 3.4× speedup

Greedy decoding, K=7, batch size 1, one TPU-v4.

Source: Leviathan et al., ICML 2023, Table 2

My results at best tested K=2

Language	Median speedup	Tokens / target call
English	**0.92×**	2.16
Hindi	**0.85×**	2.16
Urdu	**0.71×**	1.76

All three speedups below 1× indicate slowdowns.

Metric definitions and comparison notes

K = maximum draft tokens per round. 1× = baseline decoding speed.

Tokens per target call include a target correction / bonus token.

Urdu had fewer tokens per target call than Hindi for all 10 questions at each K.

Local result source: own Qwen2.5 experiments on RTX 4060 (above).

Different tasks, models and systems: not a controlled hardware comparison.

Explanation of the Topic

For translation, both models condition on the source sentence.

1 Draft

Small draft model proposes up to K tokens.

Draft tokens:
Die | Katze | sitzt | ...

→

2 Verify

Target model checks the proposed tokens together.

Draft tokens → Target
[Die] [Katze] [sitzt]

→

3 Accept / correct

Keep agreed prefix; correct first mismatch.

sitzt → schläft
Later guesses discarded

→

4 Repeat

Update context and cache; continue.

New accepted context
Die Katze schläft.

Why speed varies

Saved target work must outweigh additional drafting and verification.

Measured draft-cost ratio: $c \approx 0.67$.
Added draft work likely outweighed saved calls.

Measured limitation

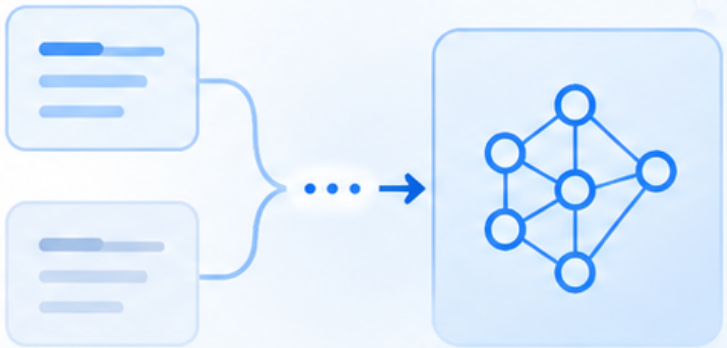
Exact speculative-output matches: 194/240 — 80.8%.

A numerical tie change was reproduced in one case.

Approximate explanation: Python overhead was not isolated.

Wrap-up / Questions

Fewer large-model calls improve speed
only when drafting is cheap enough
and the models agree.



Questions?